MODULE 10

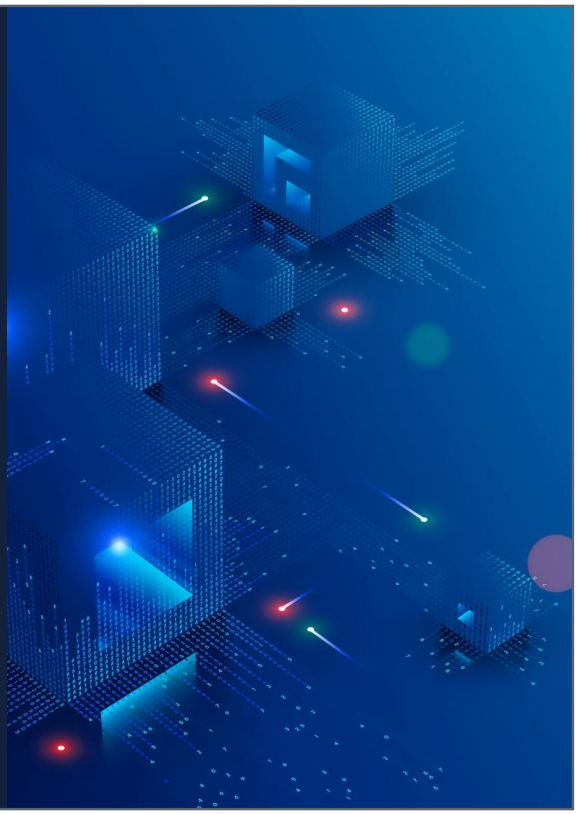
Reactive Programming with RxPY

Observables, operators, subjects and schedulers for asynchronous data streams

```
>>> rx.of(1, 2, 3).pipe(ops.map(str)).subscribe(print)
```

AIN3005 · Advanced Python Programming

Binnur Kurt, PhD · Bahçeşehir University · AI Engineering



Learning Objectives

By the end of this module, you will be able to:

1. Contrast push-based observables with pull-based iterators and plain callbacks
2. Explain the observable contract and read and draw marble diagrams
3. Create observables from values, iterables, timers and custom sources
4. Transform, filter, combine and group streams with operators in `pipe`
5. Distinguish hot from cold observables and choose a suitable subject type
6. Handle errors with `catch` and `retry` and control concurrency with schedulers
7. Build an asyncio pipeline from a WebSocket feed to RabbitMQ with RxPY

MODULE 10 · REACTIVE PROGRAMMING

Module Roadmap

01 **Why Reactive Programming**
From pulling values to reacting to them

02 **Core Concepts**
Observables, observers, subscriptions and marble diagrams

03 **Creating and Transforming Streams**
Sources, operators and their marble diagrams

04 **Subjects, Hot and Cold Observables**
Sharing one stream between many observers

05 **Errors and Schedulers**
Recovering from failures and choosing where work runs

06 **Case Studies**
WebSockets, asyncio, RxPY and RabbitMQ working together

01

Why Reactive Programming

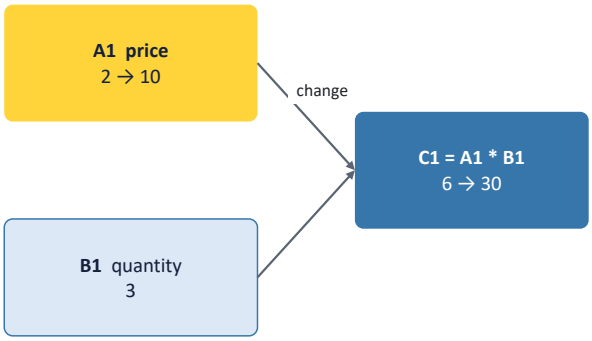
From pulling values to reacting to them

- Push vs. pull: generators and observables
- Events as streams
- The Reactive Manifesto
- RxPY and the ReactiveX family

What Is Reactive Programming?

- **Reactive programming**: programming with *asynchronous data streams*
- Values are **pushed** to your code when they happen
- Changes **propagate** automatically to everything that depends on them
- Event-driven: react to events instead of polling for them

```
>>> price, quantity = 2, 3
>>> total = price * quantity
>>> price = 10
>>> total      # imperative: not updated
6
```



A spreadsheet is reactive:
C1 is recomputed whenever
A1 or B1 changes

Pull vs. Push: Generators and Observables

Pull (Module 2): the consumer asks

```
def readings():
    yield 21.5
    yield 22.0


values = readings()
print(next(values))
print(next(values))
```

OUTPUT

```
21.5
22.0
```

Push: the producer calls you

```
import reactivex as rx

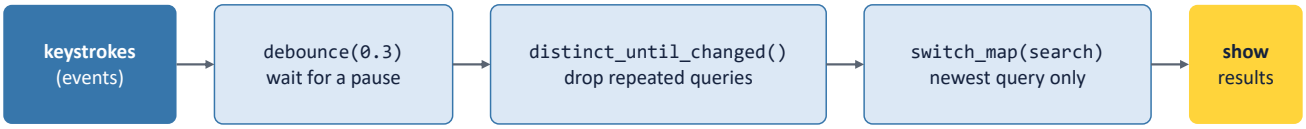
readings = rx.of(21.5, 22.0)
readings.subscribe(print)
```

OUTPUT

```
21.5
22.0
```

	Iterator / generator (pull)	Observable (push)
Who controls timing?	the consumer calls `next()`	the producer calls `on_next()`
End of data	`StopIteration` is raised	`on_completed()` is called
Failure	exception raised to the caller	`on_error(exc)` is called
Waiting for data	blocks the consumer	consumer is free until data arrives

Events as Streams



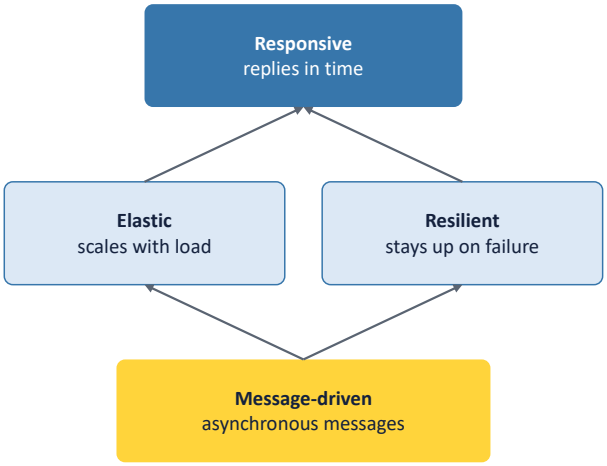
Simplicity
Compose small operators instead of nesting callbacks and flags

Scalability
Non-blocking pipelines handle high-volume, bursty event streams

Flexibility
One API for lists, timers, clicks, sockets and message queues

Typical uses: real-time UIs and dashboards, sensor/IoT data, market feeds, chat and WebSocket APIs, asynchronous service calls

The Reactive Manifesto



- A 2014 statement of properties for **reactive systems**
- Value: responsiveness; form: elastic and resilient; means: message passing
- **Reactive programming** is a *coding technique* that helps build such systems
- Module 9 brokers + Module 10 streams = both levels

Key idea
Reactive systems are an architecture; reactive programming is how you write the code inside one component.

RxPY: ReactiveX for Python

- **ReactiveX** (Rx): observer pattern + iterator pattern + functional operators
- Same model in RxJava, RxJS, Rx.NET, RxPY — skills transfer
- RxPY 4+ ships as the package `reactivex`
- Operators are functions applied with `pipe()`

TERMINAL

```
$ pip install reactivex
```

the imports used in this module

```python
import reactivex as rx
from reactivex import Observer
from reactivex import operators as ops
from reactivex.scheduler import ThreadPoolScheduler
from reactivex.subject import Subject
```

Python version note

RxPY 3 code uses `import rx`, `rx.Observable.from_` and `Observable.create`. Those names are outdated: always use the `reactivex` package.

QUIZ

Quick Check: Push or Pull?

Which of these is **push-based**?

A `next(generator)` inside a loop

B `for line in file:` reading a log file

C a `Subject` calling a subscribed handler for each WebSocket message

D `queue.get()` in a worker thread

02

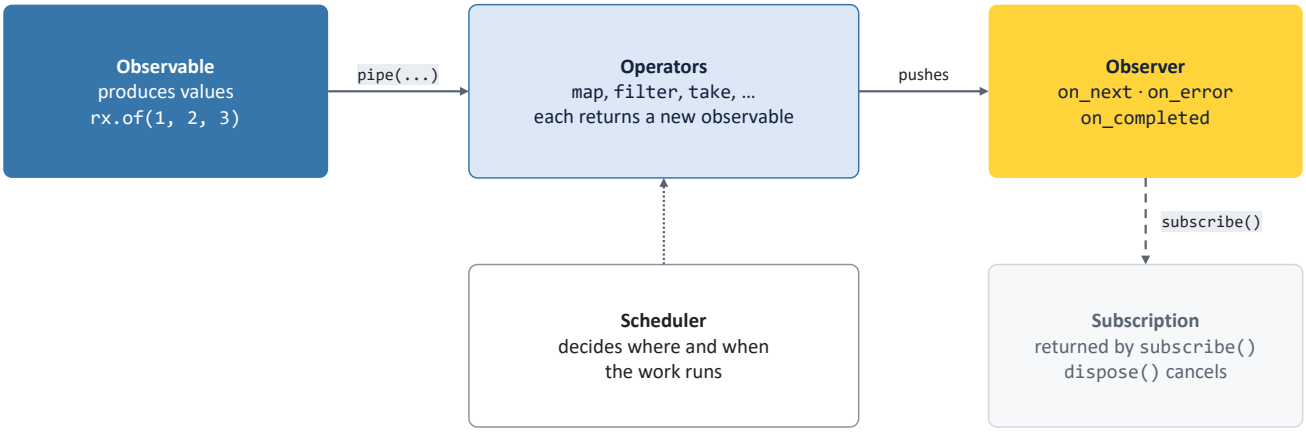
Core Concepts

Observables, observers, subscriptions and marble diagrams

- The building blocks of Rx
- Subscribing with callbacks or observer objects
- Custom sources with `create`
- The observable contract
- Marble diagrams

2 · CORE CONCEPTS

The Building Blocks of Rx




```
subscription = rx.of(1, 2, 3).pipe(ops.map(double)).subscribe(observer)
subscription.dispose()      # stop receiving values
```

2 · CORE CONCEPTS

Subscribing: Callbacks or an Observer

Three callbacks (most common)

```python
import reactivex as rx

source = rx.of(1, 2)
source.subscribe(
    on_next=lambda x: print(f"Received: {x}"),
    on_error=lambda e: print(f"Error: {e}"),
    on_completed=lambda: print("Done"),
)
```

OUTPUT

```
Received: 1
Received: 2
Done
```

An `Observer` subclass

```python
import reactivex as rx
from reactivex import Observer


class PrintObserver(Observer[int]):
    def on_next(self, value: int) -> None:
        print(f"Got: {value}")

    def on_completed(self) -> None:
        print("Sequence completed")


rx.of(1, 2).subscribe(PrintObserver())
```

OUTPUT

```
Got: 1
Got: 2
Sequence completed
```


2 · CORE CONCEPTS

Custom Sources with create

greetings.py

```python
import reactivex as rx
from reactivex.disposable import Disposable


def greetings(observer, scheduler) -> Disposable:
    """Subscribe function: runs once for every subscriber."""
    print("subscribed")
    observer.on_next("Hello")
    observer.on_next("World")
    observer.on_completed()
    return Disposable(lambda: print("cleaned up"))


source = rx.create(greetings)
source.subscribe(print)
```

OUTPUT

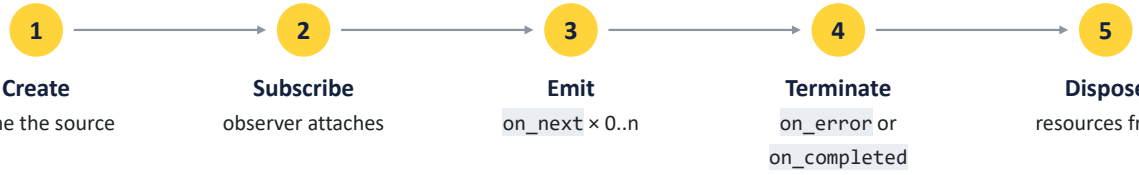
```
subscribed
Hello
World
cleaned up
```

Key idea

`rx.create(fn)` calls `fn(observer, scheduler)` on **every** `subscribe`; the returned disposable releases resources when the subscription ends.

2 · CORE CONCEPTS

The Observable Contract



Grammar of every stream

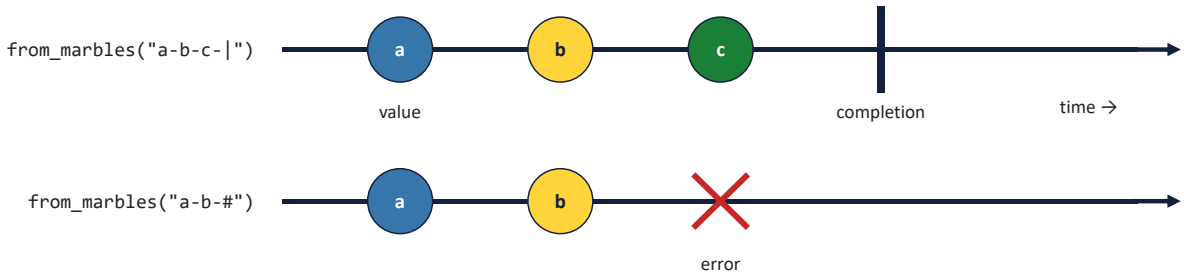
`on_next* (on_error | on_completed)?`

- At most **one** terminal event, then silence
- Calls are never concurrent (serialised)
- A stream may never terminate (e.g. a timer)

Events	Valid?
`1, 2, 3, completed`	yes
`1, 2, error`	yes
`completed` (empty stream)	yes
`1, 2, 3, …` forever	yes
`1, completed, 2`	**no**: value after the end
`error, completed`	**no**: two terminal events

2 · CORE CONCEPTS

Marble Diagrams




```python
import reactivex as rx
from reactivex import operators as ops

letters = rx.from_marbles("a-b-c-|")
print(letters.pipe(ops.to_list()).run())
```

OUTPUT

```
['a', 'b', 'c']
```

- Circle = a value (`on_next`)
- Bar `|` = `on_completed`
- Cross `#` = `on_error`
- Each `-` in text form = one time step (0.1 s)

TRACE THE CODE

Values After Completion

```
1  import reactivex as rx
2
3
4  def sensor(observer, scheduler):
5      observer.on_next(20)
6      observer.on_next(21)
7      observer.on_completed()
8      observer.on_next(99)
9      observer.on_error(RuntimeError("late"))
10
11
12 rx.create(sensor).subscribe(
13     on_next=lambda t: print(f"temp {t}"),
14     on_error=lambda e: print(f"error: {e}"),
15     on_completed=lambda: print("done"),
16 )
```

- Predict every line of output
- Which lines of `sensor` break the contract?
- What changes if line 7 is deleted?

ANSWER

Values After Completion

OUTPUT

```
temp 20
temp 21
done
```

- Lines 8–9 break the contract: events after `on_completed`
- RxPY's subscribe wrapper ignores them
- Without line 7: `temp 99`, then `error: late`



Key idea

Terminal events are final: after | the line stays empty.

QUIZ

Quick Check: The Observable Contract

Which event sequence **violates** the observable contract?

A `on_next(1)`, `on_next(2)`, `on_completed()`

B `on_completed()` only

C `on_next(1)`, `on_error(e)`, `on_next(2)`

D `on_next(1)`, `on_next(2)`, … forever

03

Creating and Transforming Streams

Sources, operators and their marble diagrams

- Creating finite and time-based streams
- `pipe` and the operator families
- Transforming and filtering: `map`, `filter`, `scan`, `reduce`, …
- Combining: `merge`, `zip`, `combine_latest`, `flat_map`
- Time and grouping: `buffer_with_count`, `window`, `debounce`

Creating Streams

Factory	Emits
`rx.of(*values)`	the given values, then completes
`rx.from_iterable(it)`	every item of an iterable (alias `from_`)
`rx.range(start, stop)`	integers, like `range`
`rx.just(value)`	exactly one value
`rx.interval(period)`	0, 1, 2, … forever
`rx.timer(due, period)`	0 after `due` (then every `period`)
`rx.create(fn)`	whatever `fn` pushes
`rx.from_callback(fn)`	the result of a callback-style API
`rx.empty()` · `rx.throw(e)`	nothing · just an error (for tests)

```python
import reactivex as rx
from reactivex import operators as ops

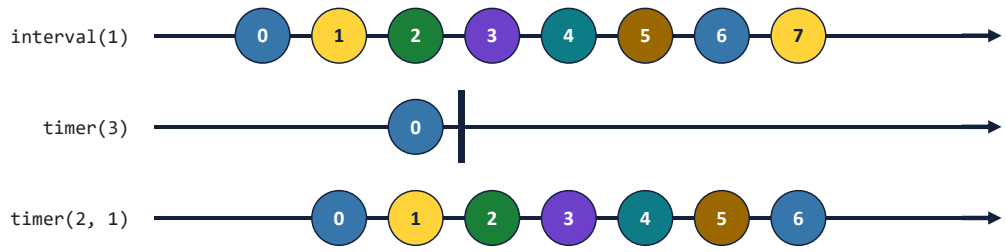

def collect(source: rx.Observable) -> list:
    """All values of a finite observable."""
    return source.pipe(ops.to_list()).run()


print(collect(rx.of("a", "b")))
print(collect(rx.from_iterable({"x": 1, "y": 2})))
print(collect(rx.range(3, 6)))
print(collect(rx.just([1, 2])))
print(collect(rx.empty()))
```

OUTPUT

```
['a', 'b']
['x', 'y']
[3, 4, 5]
[[1, 2]]
[]
```


Time-Based Sources: interval and timer




```python
import time

import reactivex as rx

ticks = rx.interval(0.05)          # 0, 1, 2, ... every 50 ms
subscription = ticks.subscribe(lambda n: print(f"tick {n}"))
time.sleep(0.18)                   # the main thread keeps going
subscription.dispose()             # stop the infinite stream
print("timer:", rx.timer(0.1).run())
```

SAMPLE OUTPUT

```
tick 0
tick 1
tick 2
timer: 0
```

Operators and pipe

```python
import reactivex as rx
from reactivex import operators as ops

even_squares = rx.range(1, 11).pipe(
    ops.filter(lambda n: n % 2 == 0),
    ops.map(lambda n: n * n),
    ops.take(3),
)
even_squares.subscribe(print)
```

OUTPUT

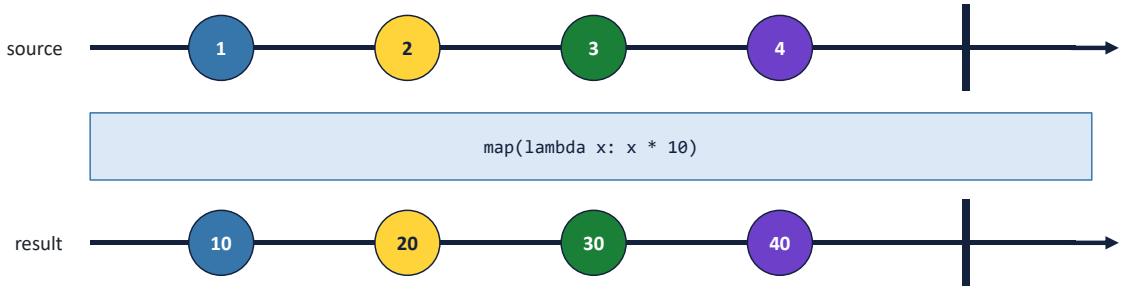
```
4
16
36
```

Family	Operators
Transforming	`map`, `scan`, `flat_map`, `buffer_with_count`
Filtering	`filter`, `take`, `skip`, `distinct`, `debounce`
Combining	`merge`, `zip`, `combine_latest`
Errors	`catch`, `retry`, `on_error_resume_next`
Utility	`do_action`, `to_list`, `share`

Key idea

`ops.map(f)` returns a function *Observable* → *Observable*; `pipe` composes such functions left to right (Module 2).

map: Transform Every Value




```python
import reactivex as rx
from reactivex import operators as ops

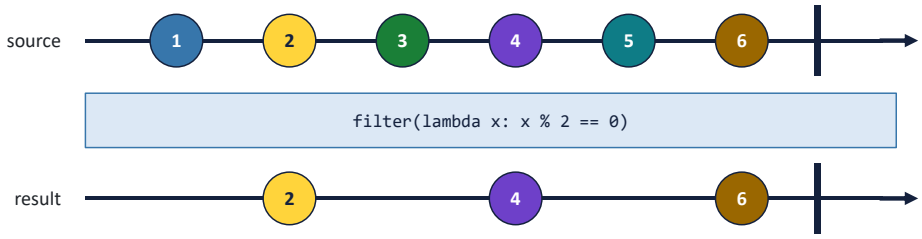
rx.of(1, 2, 3, 4).pipe(ops.map(lambda x: x * 10)).subscribe(print)

rx.from_(range(10, 16, 2)).pipe(
    ops.map_indexed(lambda x, i: f"{i}: {x * 2}"),
).subscribe(print)
```

OUTPUT

```
10
20
30
40
0: 20
1: 24
2: 28
```

filter, take, skip and distinct



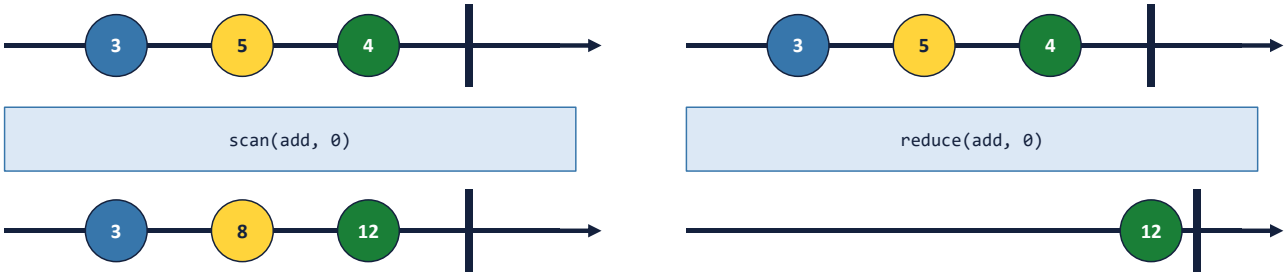

```python
import reactivex as rx
from reactivex import operators as ops

numbers = rx.of(1, 1, 2, 3, 3, 4, 5, 6)
operators = {
    "filter:": ops.filter(lambda x: x % 2 == 0),
    "take:": ops.take(3),
    "skip:": ops.skip(5),
    "distinct:": ops.distinct(),
}
for name, operator in operators.items():
    print(f"{name:9}", numbers.pipe(operator, ops.to_list()).run())
```

OUTPUT

```
filter:   [2, 4, 6]
take:     [1, 1, 2]
skip:     [4, 5, 6]
distinct: [1, 2, 3, 4, 5, 6]
```


Accumulating State: scan and reduce




```python
import reactivex as rx
from reactivex import operators as ops

sales = rx.of(3, 5, 4)
sales.pipe(ops.scan(lambda total, x: total + x, 0)).subscribe(print)
sales.pipe(ops.reduce(lambda total, x: total + x, 0)).subscribe(print)
```

OUTPUT

```
3
8
12
12
```

TRACE THE CODE

Predict the Pipeline

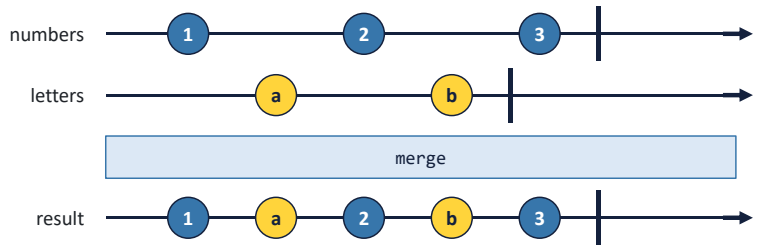
```python
import reactivex as rx
from reactivex import operators as ops

rx.range(1, 8).pipe(
    ops.filter(lambda n: n % 3 != 0),
    ops.map(lambda n: n * 10),
    ops.scan(lambda acc, n: acc + n, 0),
    ops.skip(1),
    ops.take(3),
).subscribe(print, on_completed=lambda: print("done"))
```

- Write the values after **each** operator
- Predict the complete output
- Is the value 7 from `range` ever used?

3 · CREATING AND TRANSFORMING STREAMS

merge: Interleaving Streams




```python
import reactivex as rx
from reactivex import operators as ops

numbers = rx.range(1, 5)
letters = rx.from_("abcde")
numbers.pipe(ops.merge(letters)).subscribe(
    lambda value: print(value, end=" "))
```

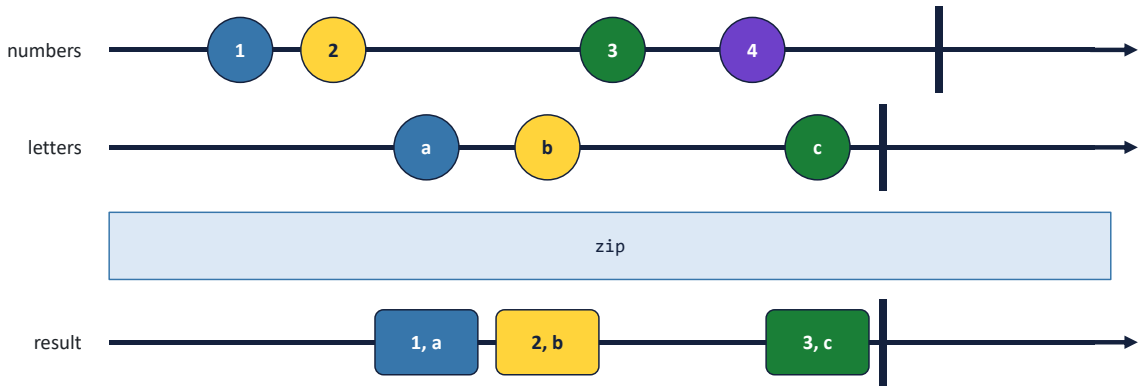
OUTPUT

```
1 a b c d e 2 3 4
```

The spacetime of Rx

Without time between events, the order depends on the scheduler. Rx only guarantees the order **within** each source.

zip: Pairing Values by Position




```python
import reactivex as rx

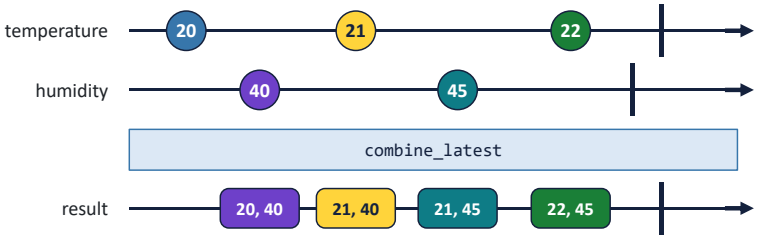
names = rx.of("Ada", "Alan", "Grace")
scores = rx.of(91, 78, 95, 60)
rx.zip(names, scores).subscribe(print)
```

OUTPUT

```
('Ada', 91)
('Alan', 78)
('Grace', 95)
```

- n-th value of each source → one tuple
- Waits for the slowest source
- Completes when a finished source has no queued values

combine_latest: Latest From Each Stream




```python
import reactivex as rx
from reactivex.subject import Subject

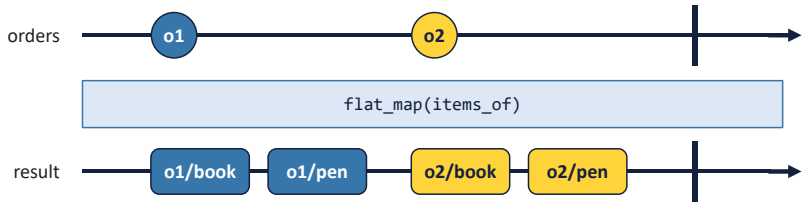
temperature = Subject[int]()
humidity = Subject[int]()
rx.combine_latest(temperature, humidity).subscribe(print)

temperature.on_next(20)          # no humidity yet: nothing
humidity.on_next(40)
temperature.on_next(21)
humidity.on_next(45)
temperature.on_next(22)
```

OUTPUT

```
(20, 40)
(21, 40)
(21, 45)
(22, 45)
```

flat_map: One Value, Many Values




```python
import reactivex as rx
from reactivex import operators as ops


def items_of(order: str) -> rx.Observable[str]:
    """Each order expands into a stream of its items."""
    return rx.of(f"{order}/book", f"{order}/pen")

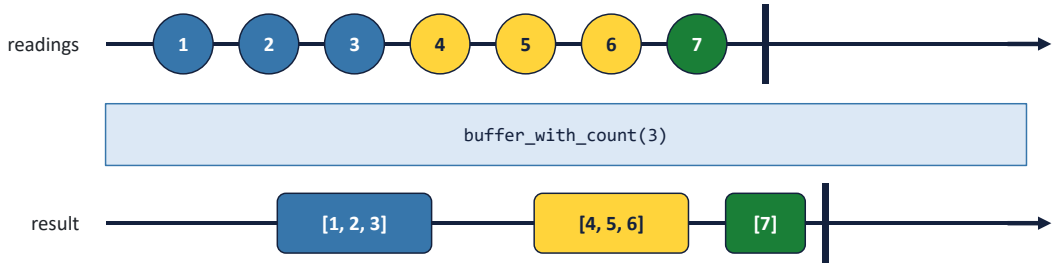

orders = rx.of("o1", "o2")
orders.pipe(ops.flat_map(items_of)).subscribe(print)
```

OUTPUT

```
o1/book
o1/pen
o2/book
o2/pen
```

`flat_map` = `map` + `merge` · `concat_map` keeps inner streams in order · `switch_map` keeps only the newest

Grouping: buffer_with_count and window



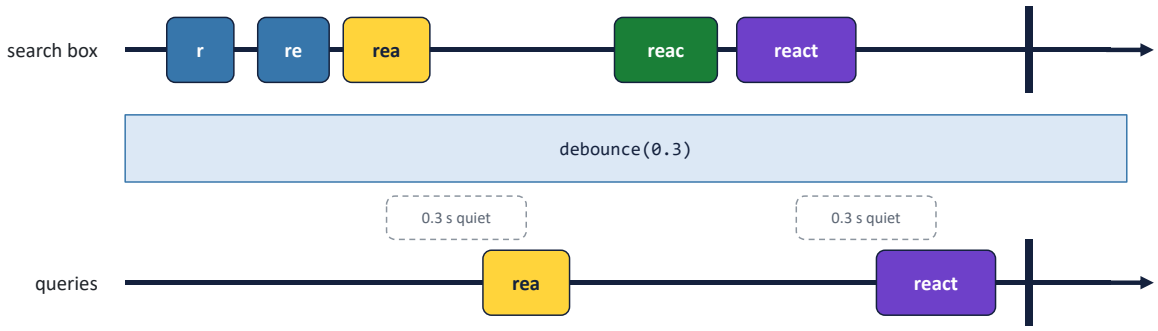

```python
import reactivex as rx
from reactivex import operators as ops

readings = rx.range(1, 8)
readings.pipe(ops.buffer_with_count(3)).subscribe(print)
readings.pipe(
    ops.window_with_count(3),
    ops.flat_map(lambda window: window.pipe(ops.sum())),
).subscribe(print)
```

OUTPUT

```
[1, 2, 3]
[4, 5, 6]
[7]
6
15
7
```

debounce: Waiting for a Pause




```python
import reactivex as rx
from reactivex import operators as ops

typed = {"a": "r", "b": "re", "c": "rea", "d": "reac", "e": "react"}
keys = rx.from_marbles("a-b-c----d-e----|", timespan=0.02, lookup=typed)
queries = keys.pipe(ops.debounce(0.06), ops.to_list())
print(queries.run())
```

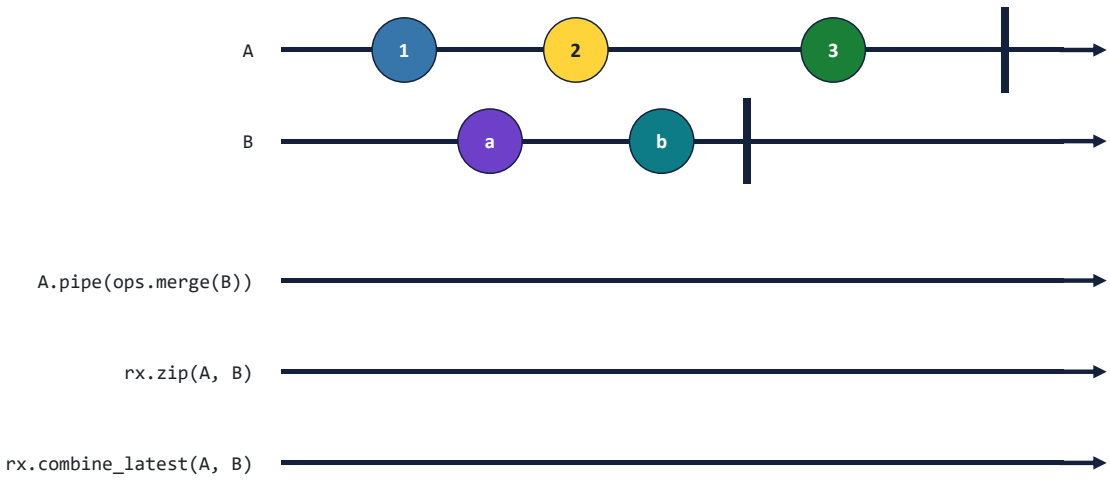
SAMPLE OUTPUT

```
['rea', 'react']
```


TRACE THE CODE

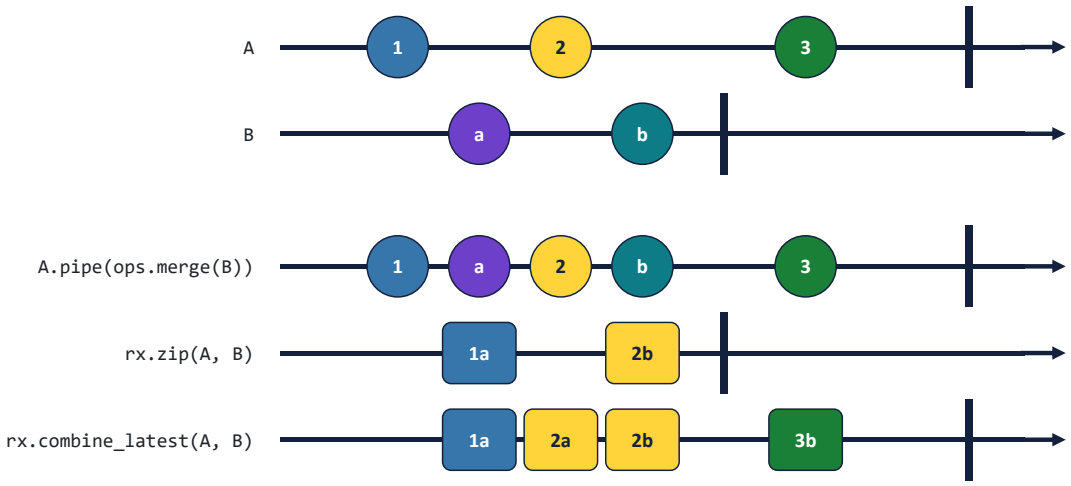
Draw the Output Marbles

Inputs `A` and `B` are given. Draw the three output lines, including completion (write a pair as `1a`).



ANSWER

Draw the Output Marbles



`merge` forwards everything · `zip` completes with `B` (nothing left to pair) · `combine_latest` emits on every event once both have a value

QUIZ

Quick Check: Pick the Operator

A sensor emits a reading every 10 ms; a calibration offset changes a few times per hour. Which operator pairs **each reading with the current offset**?

- **A** `zip(readings, offsets)`
- **B** `merge(readings, offsets)`
- **C** `combine_latest(readings, offsets)`
- **D** `concat(readings, offsets)`

04

Subjects, Hot and Cold Observables

Sharing one stream between many observers

- `Subject`: observable and observer at once
- Cold vs. hot observables
- `BehaviorSubject`, `ReplaySubject`, `AsyncSubject`
- Multicasting with `publish` and `share`

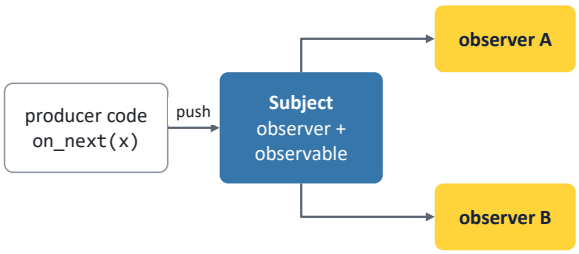
Subject: Observable and Observer at Once

```python
from reactivex.subject import Subject

stream = Subject[int]()
stream.on_next(41)          # nobody listening: lost
subscription = stream.subscribe(
    lambda x: print(f"Got: {x}"))
stream.on_next(42)
subscription.dispose()
stream.on_next(43)          # unsubscribed
```

OUTPUT

```
Got: 42
```



Key idea

A subject forwards each value to **all** current subscribers.

Cold vs. Hot Observables

	Cold	Hot
Producer	created by each `subscribe`	exists independently of subscribers
Late subscriber	gets the whole sequence from the start	misses values emitted before
Examples	`of`, `range`, `interval`, `create`, an HTTP call	`Subject`, clicks, WebSocket feed, sensor
Analogy	a video on demand	a live broadcast

```python
import reactivex as rx
from reactivex import operators as ops

cold = rx.of(1, 2).pipe(ops.do_action(lambda v: print("produce", v)))
cold.subscribe(lambda v: print("  A got", v))
cold.subscribe(lambda v: print("  B got", v))
```

OUTPUT

```
produce 1
  A got 1
produce 2
  A got 2
produce 1
  B got 1
produce 2
  B got 2
```


BehaviorSubject and ReplaySubject

BehaviorSubject: the current value

```python
from reactivex.subject import BehaviorSubject

temperature = BehaviorSubject(20.0)
temperature.subscribe(lambda t: print(f"A: {t}"))
temperature.on_next(21.5)
temperature.subscribe(lambda t: print(f"B: {t}"))
print("current:", temperature.value)
```

OUTPUT

```
A: 20.0
A: 21.5
B: 21.5
current: 21.5
```

ReplaySubject: the last *n* values

```python
from reactivex.subject import ReplaySubject

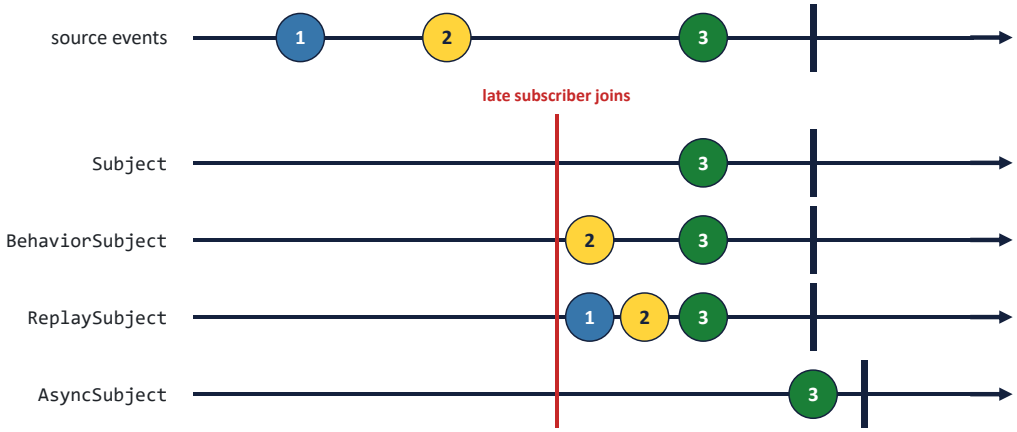
chat = ReplaySubject(buffer_size=2)
for text in ["hi", "how are you?", "fine"]:
    chat.on_next(text)
chat.subscribe(lambda t: print(f"late: {t}"))
```

OUTPUT

```
late: how are you?
late: fine
```

> **Note**
>
> `AsyncSubject` emits only the **last** value, and only when it completes — like a future.

What Does a Late Subscriber Receive?



Key idea

The subject type is a policy for **late subscribers**: nothing, the latest value, a history, or only the final value.

Multicasting with publish and share

```python
import reactivex as rx
from reactivex import operators as ops

source = rx.of(1, 2).pipe(
    ops.do_action(lambda v: print("produce", v)),
    ops.publish(),                # a ConnectableObservable
)
source.subscribe(lambda v: print("  A got", v))
source.subscribe(lambda v: print("  B got", v))
source.connect()                  # start one shared run
```

OUTPUT

```
produce 1
  A got 1
  B got 1
produce 2
  A got 2
  B got 2
```



Tip

`share()` = `publish()` + `ref_count()`: it connects on the first subscriber and disconnects after the last one leaves.

TRACE THE CODE

Who Receives What?

```python
from reactivex.subject import BehaviorSubject, ReplaySubject, Subject

plain = Subject[str]()
latest = BehaviorSubject("idle")
replay = ReplaySubject[str](buffer_size=2)
for subject in (plain, latest, replay):
    subject.on_next("start")
    subject.on_next("run")

plain.subscribe(lambda s: print("plain:", s))
latest.subscribe(lambda s: print("latest:", s))
replay.subscribe(lambda s: print("replay:", s))
for subject in (plain, latest, replay):
    subject.on_next("stop")
```

Predict the output, line by line. When is each line printed?

QUIZ

Quick Check: Hot or Cold?

Which of these observables is **hot**?

A `rx.of(1, 2, 3)`

B `rx.interval(1.0)`

C a `Subject` that forwards messages from a WebSocket

D `rx.from_iterable(open("log.txt"))`

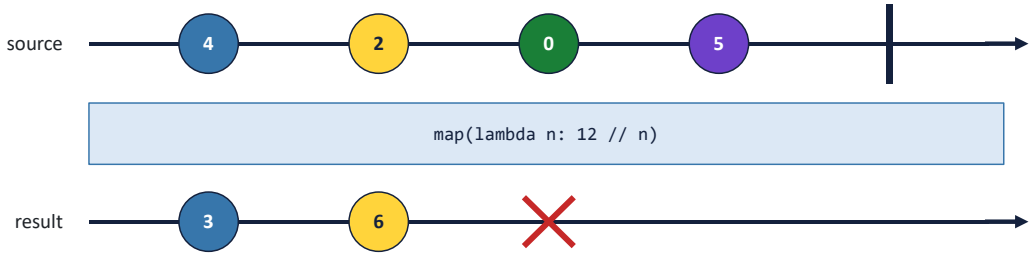
05

Errors and Schedulers

Recovering from failures and choosing where work runs

- Errors terminate a stream
- `catch`, `on_error_resume_next` and `retry`
- Schedulers and thread pools (Module 5)
- RxPY with asyncio (Module 5)

Errors Terminate the Stream




```python
import reactivex as rx
from reactivex import operators as ops

rx.of(4, 2, 0, 5).pipe(
    ops.map(lambda n: 12 // n),
).subscribe(
    on_next=print,
    on_error=lambda error: print(f"on_error: {error!r}"),
    on_completed=lambda: print("done"),
)
```

OUTPUT

```
3
6
on_error: ZeroDivisionError('division by zero')
```

Recovering: catch and on_error_resume_next

catch: switch to a fallback stream

```python
import reactivex as rx
from reactivex import operators as ops


def fallback(error: Exception,
             source: rx.Observable) -> rx.Observable:
    print(f"recovering from {type(error).__name__}")
    return rx.of(-1)


rx.of(4, 2, 0, 5).pipe(
    ops.map(lambda n: 12 // n),
    ops.catch(fallback),
).subscribe(print)
```

OUTPUT

```
3
6
recovering from ZeroDivisionError
-1
```

on_error_resume_next: continue

```python
import reactivex as rx
from reactivex import operators as ops

bad = rx.concat(rx.of(1, 2),
                rx.throw(ValueError()))
spare = rx.of(10, 20)
resumed = bad.pipe(
    ops.on_error_resume_next(spare))
resumed.subscribe(print)
```

OUTPUT

```
1
2
10
20
```

Watch out

Recovery **replaces** the failed stream; it cannot resume it: the value 5 is lost.

retry: Subscribe Again After a Failure

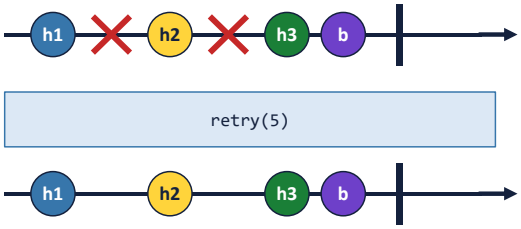
```python
import itertools

import reactivex as rx
from reactivex import operators as ops

attempts = itertools.count(1)


def flaky_fetch(observer, scheduler):
    attempt = next(attempts)
    observer.on_next(f"header {attempt}")
    if attempt < 3:
        observer.on_error(ConnectionError())
    else:
        observer.on_next("body")
        observer.on_completed()


rx.create(flaky_fetch).pipe(ops.retry(5)).subscribe(print)
```



OUTPUT

```
header 1
header 2
header 3
body
```

Watch out

`retry` re-runs a **cold** source: values sent before the failure arrive again.

FIND THE BUG

Find the Bug: One Bad Message Stops Everything

```python
import json

import reactivex as rx
from reactivex import operators as ops

messages = rx.of('{"p": "10.5"}', "garbage", '{"p": "11.0"}')
messages.pipe(
    ops.map(json.loads),
    ops.map(lambda trade: float(trade["p"])),
).subscribe(
    on_next=print,
    on_error=lambda e: print(f"stream failed: {type(e).__name__}"),
)
```

OUTPUT

```
10.5
stream failed: JSONDecodeError
```

- Why is `11.0` never printed?
- Change the pipeline so that bad messages are skipped and the stream keeps running

ANSWER

Find the Bug: One Bad Message Stops Everything

```python
def safe_price_stream(message: str) -> rx.Observable[float]:
    """One message -> a price, or an empty stream if it is bad."""
    return rx.of(message).pipe(
        ops.map(json.loads),
        ops.map(lambda trade: float(trade["p"])),
        ops.catch(rx.empty()),
    )


messages.pipe(ops.flat_map(safe_price_stream)).subscribe(print)
```

OUTPUT

```
10.5
11.0
```

Key idea

Isolate each item: an error inside the inner stream ends only that inner stream. A plain function that returns `None` for bad input plus `filter` works too.

Schedulers: Where and When Work Runs

Scheduler	Runs work …	Typical use
`CurrentThreadScheduler`	queued on the calling thread (trampoline)	default for `of`, `range`, `from_iterable`
`ImmediateScheduler`	immediately, on the calling thread	rarely needed directly
`TimeoutScheduler`	on `threading.Timer` threads	default for `interval`, `timer`, `debounce`
`NewThreadScheduler`	on a new thread per subscription	long-running, isolated work
`ThreadPoolScheduler(n)`	on a pool of n worker threads	blocking I/O in parallel (Module 5)
`EventLoopScheduler`	on one dedicated background thread	serialising work off the main thread
`AsyncIOScheduler(loop)`	on the asyncio event loop	asyncio and FastAPI applications
`HistoricalScheduler`	in virtual time	fast, deterministic tests

Pass `scheduler=` to a factory or time-based operator, or to `subscribe(..., scheduler=s)`; move work with `subscribe_on` and `observe_on`

Blocking Work on a ThreadPoolScheduler

```python
import threading
import time

import reactivex as rx
from reactivex import operators as ops
from reactivex.scheduler import ThreadPoolScheduler


def fetch(url: str) -> str:
    time.sleep(0.1)                          # blocking I/O
    return f"{url} on {threading.current_thread().name}"

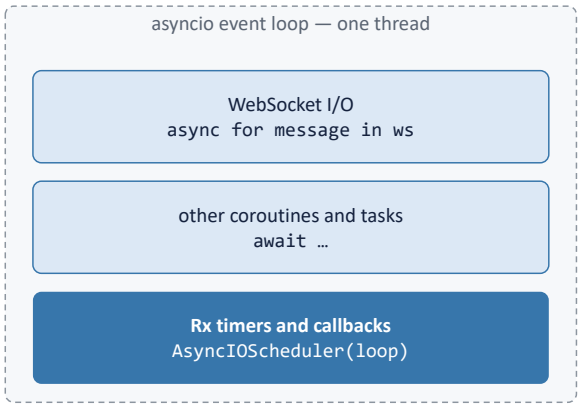

pool = ThreadPoolScheduler(4)
pages = rx.of("a.html", "b.html", "c.html").pipe(
    ops.flat_map(lambda url: rx.of(url).pipe(
        ops.map(fetch), ops.subscribe_on(pool))),
    ops.to_list(),
)
print(*pages.run(), sep="\n")
```

SAMPLE OUTPUT

```
a.html on ThreadPoolExecutor-0_0
b.html on ThreadPoolExecutor-0_1
c.html on ThreadPoolExecutor-0_2
```

`subscribe_on(s)`: where the **source** runs · `observe_on(s)`: where **downstream** operators and observers run

RxPY and asyncio



Need	Tool
run Rx time-based work on the loop	`AsyncIOScheduler(loop)`
awaitable → observable	`rx.from_future(task)`
observable → awaitable	`await observable` or `ops.to_future()`

Key idea

asyncio (Module 5) moves bytes without blocking; RxPY describes what to do with them. Keep both on **one** event loop.

AsyncIOScheduler in Practice

rx_asyncio.py

```python
import asyncio

import reactivex as rx
from reactivex import operators as ops
from reactivex.scheduler.eventloop import AsyncIOScheduler


async def main() -> None:
    scheduler = AsyncIOScheduler(asyncio.get_running_loop())
    ticks = rx.interval(0.01, scheduler=scheduler).pipe(ops.take(3))
    print(await ticks.pipe(ops.to_list()))       # observable -> await

    slow = asyncio.ensure_future(asyncio.sleep(0.03, result="BTC"))
    fast = asyncio.ensure_future(asyncio.sleep(0.01, result="ETH"))
    prices = rx.merge(rx.from_future(slow), rx.from_future(fast))
    print(await prices.pipe(ops.to_list()))      # future -> observable


asyncio.run(main())
```

SAMPLE OUTPUT

```
[0, 1, 2]
['ETH', 'BTC']
```

QUIZ

Quick Check: Pick the Scheduler

A FastAPI service receives WebSocket messages and groups them with `buffer_with_time(30)`. Which scheduler should `buffer_with_time` use?

A. the default `TimeoutScheduler`

B. `AsyncIOScheduler` on the running loop

C. `NewThreadScheduler`

D. `ImmediateScheduler`

06

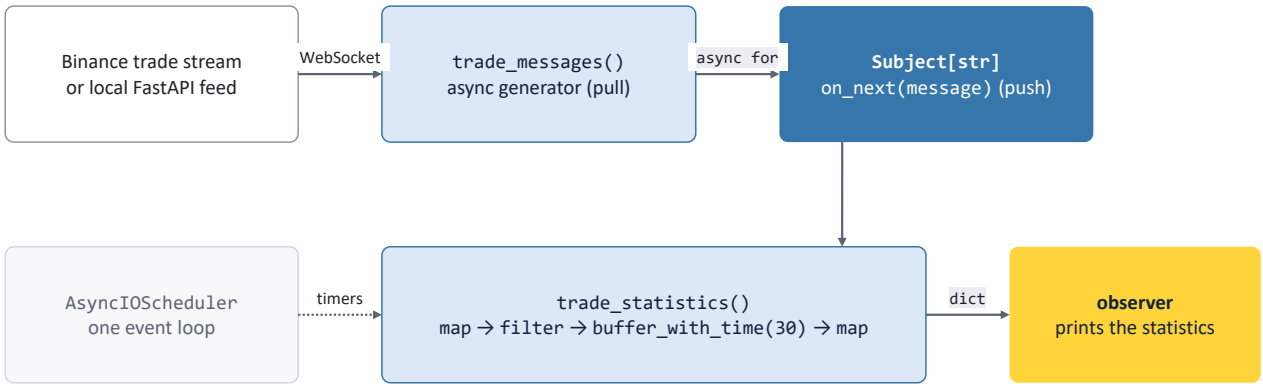
Case Studies

WebSockets, asyncio, RxPY and RabbitMQ working together

- Case study 1: live trade statistics from a WebSocket feed
- A local FastAPI trade feed for offline testing
- Case study 2: from WebSocket to RabbitMQ (Module 9)
- What we fixed in the original code

6 · CASE STUDIES

Case Study 1: Live Trade Statistics



trade_stats.py — configuration

```
STREAM_URL = "wss://stream.binance.com:9443/stream?streams="
PAIRS = ["btcusdt@trade"]          # one instrument per statistic
WINDOW_SECONDS = 30.0
```


6 · CASE STUDIES

Parsing and Summarising Trades

trade_stats.py — pure helpers

```
import json
from statistics import fmean


def parse_price(message: str) -> float | None:
    """Trade price from a message, or None."""
    try:
        return float(json.loads(message)["data"]["p"])
    except (KeyError, TypeError, ValueError):
        return None


def summarize(prices: list[float]) -> dict[str, float]:
    """Count, mean, min and max of one non-empty window."""
    return {
        "count": len(prices),
        "average": round(fmean(prices), 2),
        "min": min(prices),
        "max": max(prices),
    }
```

```
trade = '{"data": {"p": "64210.5"}}'
print(parse_price(trade))
print(parse_price('{"id": 1}'))
print(parse_price("not json"))
stats = summarize([10.0, 12.5, 11.0])
print(stats["average"], stats["max"])
```

OUTPUT

```
64210.5
None
None
11.17 12.5
```

Tip

Pure functions: testable with plain `pytest` (Module 3), no network or Rx needed.

6 · CASE STUDIES

Imports and the WebSocket Source

trade_stats.py — I/O

```python
import asyncio
from collections.abc import AsyncIterator

import websockets
from reactivex import Observable
from reactivex import operators as ops
from reactivex.scheduler.eventloop import AsyncIOScheduler
from reactivex.subject import Subject


async def trade_messages(pairs: list[str]) -> AsyncIterator[str]:
    """Yield raw JSON messages from the combined trade stream."""
    url = STREAM_URL + "/".join(pairs)
    async with websockets.connect(url) as websocket:
        async for message in websocket:
            yield message
```

Python version note

RxPY 3 imports (`from rx import operators`, `rx.scheduler.eventloop`) replaced by the `reactivex` package.

6 · CASE STUDIES

The Statistics Pipeline

trade_stats.py — pipeline

```python
def trade_statistics(messages: Observable[str],
                     scheduler: AsyncIOScheduler) -> Observable[dict]:
    """Price statistics for every 30-second window of trades."""
    return messages.pipe(
        ops.map(parse_price),
        ops.filter(lambda price: price is not None),
        ops.buffer_with_time(WINDOW_SECONDS, scheduler=scheduler),
        ops.filter(lambda window: len(window) > 0),
        ops.map(summarize),
    )
```



6 · CASE STUDIES

Running the Statistics Client

trade_stats.py — entry point

```python
async def main() -> None:
    scheduler = AsyncIOScheduler(asyncio.get_running_loop())
    messages: Subject[str] = Subject()
    subscription = trade_statistics(messages, scheduler).subscribe(
        on_next=lambda stats: print(f"30 s window: {stats}"),
        on_error=lambda error: print(f"Error: {error}"),
    )
    try:
        async for message in trade_messages(PAIRS):
            messages.on_next(message)
    finally:
        subscription.dispose()


if __name__ == "__main__":
    asyncio.run(main())
```

SAMPLE OUTPUT (LIVE PRICES VARY)

```
30 s window: {'count': 1874, 'average': 64212.37, 'min': 64180.0, 'max': 64251.2}
```


6 · CASE STUDIES

A Local Trade Feed with FastAPI

feed.py

```python
import asyncio
import random

from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()


@app.websocket("/stream")
async def trade_feed(websocket: WebSocket) -> None:
    """Send a simulated BTCUSDT trade every 100 ms."""
    await websocket.accept()
    price = 64_000.0
    try:
        while True:
            price += random.uniform(-5.0, 5.0)
            trade = {"s": "BTCUSDT", "p": f"{price:.2f}"}
            await websocket.send_json({"data": trade})
            await asyncio.sleep(0.1)
    except WebSocketDisconnect:
        pass
```

TERMINAL

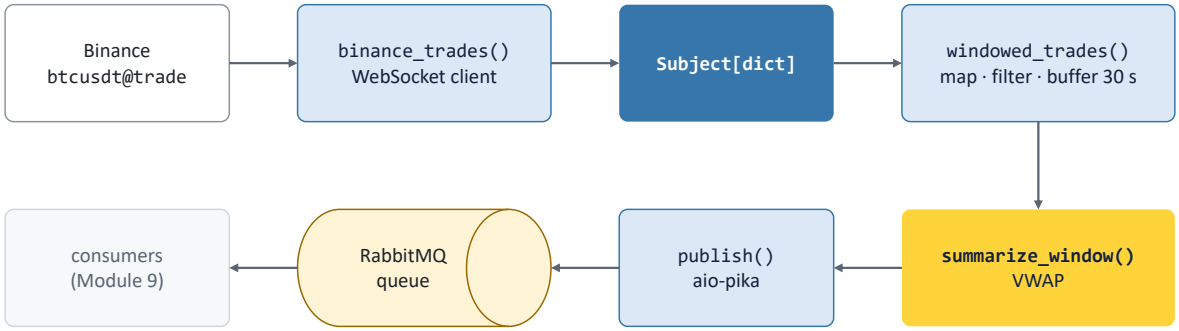
```
$ pip install fastapi uvicorn
$ uvicorn feed:app --port 8000
```

In `trade_stats.py` set `STREAM_URL` to `"ws://localhost:8000/stream?streams="`

Key idea

Server and client are two observers talking over a WebSocket: FastAPI pushes, the client's subject receives.

Case Study 2: From WebSocket to RabbitMQ



trade_pipeline.py — configuration

```
BINANCE_WS_URL = "wss://stream.binance.com:9443/ws/btcusdt@trade"
RABBITMQ_URL = "amqp://localhost/"
QUEUE_NAME = "filtered_trades"
MIN_PRICE = 50_000.0              # only "expensive" trades
WINDOW_SECONDS = 30.0
```


Imports and the Trade Feed

trade_pipeline.py — imports and source

```
import asyncio
import json

import aio_pika
import reactivex as rx
import websockets
from aio_pika.abc import AbstractExchange
from reactivex import Observable
from reactivex import operators as ops
from reactivex.scheduler.eventloop import AsyncIOScheduler
from reactivex.subject import Subject


async def binance_trades(trades: Subject[dict]) -> None:
    """Push every trade message from Binance into the subject."""
    async with websockets.connect(BINANCE_WS_URL) as websocket:
        async for message in websocket:
            trades.on_next(json.loads(message))
```

Filtering and Windowing Trades

trade_pipeline.py — windowing

```python
def to_trade(raw: dict) -> dict:
    """Keep the fields we need, converted to proper types."""
    return {"symbol": raw["s"], "price": float(raw["p"]),
            "quantity": float(raw["q"]), "timestamp": raw["T"]}


def windowed_trades(trades: Observable[dict],
                    scheduler: AsyncIOScheduler) -> Observable[list]:
    """Expensive trades, grouped into 30-second windows."""
    return trades.pipe(
        ops.map(to_trade),
        ops.filter(lambda trade: trade["price"] > MIN_PRICE),
        ops.buffer_with_time(WINDOW_SECONDS, scheduler=scheduler),
        ops.filter(lambda window: len(window) > 0),
    )
```

Tip

Convert first, then filter: each price string is parsed once, and empty windows never reach the VWAP calculation.

Volume-Weighted Average Price

trade_pipeline.py — summary

```python
def summarize_window(trades: list[dict]) -> dict:
    """Volume-weighted average price (VWAP) of one window."""
    volume = sum(t["quantity"] for t in trades)
    turnover = sum(t["price"] * t["quantity"] for t in trades)
    return {
        "symbol": trades[0]["symbol"],
        "vwap": round(turnover / volume, 2),
        "total_quantity": round(volume, 6),
        "window_start": trades[0]["timestamp"],
        "window_end": trades[-1]["timestamp"],
    }
```

```python
window = [
    {"symbol": "BTCUSDT", "price": 64000.0,
     "quantity": 0.5, "timestamp": 1},
    {"symbol": "BTCUSDT", "price": 64100.0,
     "quantity": 1.5, "timestamp": 2},
]
summary = summarize_window(window)
print(summary["vwap"], summary["total_quantity"])
```

OUTPUT

```
64075.0 2.0
```

VWAP = Σ(price × quantity) / Σ quantity: large trades weigh more than small ones

Publishing Summaries with aio-pika

trade_pipeline.py — sink

```python
async def publish(exchange: AbstractExchange, summary: dict) -> dict:
    """Send one summary as a persistent JSON message."""
    message = aio_pika.Message(
        body=json.dumps(summary).encode(),
        content_type="application/json",
        delivery_mode=aio_pika.DeliveryMode.PERSISTENT,
    )
    await exchange.publish(message, routing_key=QUEUE_NAME)
    return summary


def publish_summaries(windows: Observable[list],
                      exchange: AbstractExchange) -> Observable[dict]:
    """Summarise each window, publish it, emit what was sent."""
    return windows.pipe(
        ops.map(summarize_window),
        ops.flat_map(lambda summary: rx.from_future(
            asyncio.ensure_future(publish(exchange, summary)))),
    )
```


Running the RabbitMQ Pipeline

trade_pipeline.py — entry point

```python
async def main() -> None:
    scheduler = AsyncIOScheduler(asyncio.get_running_loop())
    trades: Subject[dict] = Subject()
    connection = await aio_pika.connect_robust(RABBITMQ_URL)
    async with connection:
        channel = await connection.channel()
        await channel.declare_queue(QUEUE_NAME, durable=True)
        windows = windowed_trades(trades, scheduler)
        sent = publish_summaries(windows, channel.default_exchange)
        subscription = sent.subscribe(
            on_next=lambda summary: print(f"Sent: {summary}"),
            on_error=lambda error: print(f"Error: {error}"))
        try:
            await binance_trades(trades)
        finally:
            subscription.dispose()


if __name__ == "__main__":
    asyncio.run(main())
```

Consuming the Summaries with pika

vwap_consumer.py

```python
import json

import pika

QUEUE_NAME = "filtered_trades"


def handle(channel, method, properties, body: bytes) -> None:
    summary = json.loads(body)
    print(f"VWAP {summary['vwap']} ({summary['total_quantity']} BTC)")
    channel.basic_ack(delivery_tag=method.delivery_tag)


if __name__ == "__main__":
    params = pika.ConnectionParameters("localhost")
    with pika.BlockingConnection(params) as connection:
        channel = connection.channel()
        channel.queue_declare(queue=QUEUE_NAME, durable=True)
        channel.basic_consume(QUEUE_NAME, on_message_callback=handle)
        channel.start_consuming()
```


What We Fixed in the Original Code

Original	Problem	Revised
`from rx import …` (RxPY 3)	unmaintained package	`reactivex` 4+ imports
`asyncio.get_event_loop()`	deprecated outside a running loop	`get_running_loop()` in `main()`
`pika.BlockingConnection` in the event loop	blocks the loop; missed heartbeats	`aio-pika` + `flat_map(from_future)`
connection and subject at import time	side effects on import; hard to test	created in `main()`, passed as arguments
BTC and ETH in one window	the average mixes two instruments	one symbol (or `group_by`)
`except Exception`, `if … else None` per field	hides bugs; noisy results	specific exceptions; filter empty windows
non-durable queue, no consumer	messages lost on restart	durable queue, persistent messages, manual ack

MINI LAB

Practice Problems

Moving-average alarm

Readings arrive every second. Emit `"ALARM"` when the average of the last 3 exceeds 30. Hint: `buffer_with_count(3, 1)`, `distinct_until_changed`

Type-ahead search

Debounce keystrokes by 0.3 s, skip repeated queries and cancel stale searches with `switch_map`

Per-symbol statistics

Extend case study 1 to BTC and ETH with `group_by` + `flat_map`, one summary per symbol and window

Test with virtual time

Test `trade_statistics` in pytest with `HistoricalScheduler` and `from_marbles`: no network, no waiting

MODULE 10 · SUMMARY

Key Takeaways

- ✓ Reactive programming is push-based: observables emit `on_next*` then at most one `on_error` or `on_completed`
- ✓ Operators in `pipe` build lazy pipelines; marble diagrams show what they do with time
- ✓ `merge`, `zip`, `combine_latest` and `flat_map` differ in how they line values up in time
- ✓ Cold observables run per subscriber; subjects and `publish`/`share` make one hot, shared stream
- ✓ An error terminates a stream: recover with `catch`/`retry`, or isolate items with `flat_map`
- ✓ Schedulers decide where work runs: thread pools for blocking I/O, `AsyncIOScheduler` inside asyncio

NEXT UP Module 11 — Python Performance Tuning and Optimization